

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ

ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ

ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

по расчетно-графической работе
по дисциплине «Программирование»

Выполнил:

студент группы АБ-522

Никулин Р. Д.

Проверил:

(должность)

Архипова А. Б.

Новосибирск 2026

АННОТАЦИЯ

Настоящая пояснительная записка составлена по ГОСТ 19.404-79 и относится к программе «Multi-Algo Cryptotool», разработанной в ходе расчетно-графической работы по дисциплине «Программирование» в соответствии с техническим заданием, оформленным по ГОСТ 19.201-78.

Программа шифрует и расшифровывает данные произвольного типа алгоритмом, который пользователь выбирает при запуске, и создает ключи для этих алгоритмов. Реализации алгоритмов вынесены в отдельные динамически загружаемые библиотеки и подключаются во время выполнения через двоичный интерфейс языка С.

В документе приведены назначение и область применения программы, постановка задачи, описание алгоритма и функционирования программы вместе со схемой архитектуры, обоснование выбранного способа организации входных и выходных данных, обоснование состава технических и программных средств, ожидаемые технико-экономические показатели и перечень использованных источников.

СОДЕРЖАНИЕ

1 Введение	4
2 Назначение и область применения	4
3 Технические характеристики	6
3.1 Постановка задачи на разработку программы	6
3.2 Описание алгоритма и функционирования программы	8
3.2.1 Состав программы и схема архитектуры	8
3.2.2 Двоичный интерфейс модулей шифрования	11
3.2.3 Поточковая обработка данных	13
3.2.4 Реализованные алгоритмы шифрования	14
3.2.5 Создание ключей и защита данных в оперативной памяти	16
3.2.6 Обработка ошибок и коды возврата	17
3.3 Описание и обоснование выбора метода организации входных и выходных данных	17
3.4 Описание и обоснование выбора состава технических и программных средств	20
4 Ожидаемые технико-экономические показатели	23
5 Источники, использованные при разработке	25

1 ВВЕДЕНИЕ

Наименование программы: «Multi-Algo Cryptotool». Исполняемый файл носит имя `cryptum`, под этим же именем программа вызывается из командной строки.

Программа выполняет криптографическое преобразование данных произвольного типа и создает ключи шифрования. Алгоритм и режим работы задаются аргументами командной строки. Реализации алгоритмов размещены в отдельных динамически загружаемых библиотеках, по одной библиотеке на алгоритм, и подключаются во время выполнения. Исполняемый файл не содержит ни одной реализации алгоритма.

Область применения программы ограничена учебным процессом кафедры защиты информации. Программа служит наглядным пособием при изучении криптографических алгоритмов и приемов построения программ с модульной архитектурой. Реализованные шифры относятся к классическим, вскрываются частотным анализом и не обеспечивают стойкости, достаточной для защиты реальной информации, поэтому применение программы за пределами учебных задач не предполагается.

Настоящая пояснительная записка составлена по ГОСТ 19.404-79 [2] и входит в комплект программной документации на изделие наряду с техническим заданием, оформленным по ГОСТ 19.201-78 [1], текстом программы и руководством оператора.

2 НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

Функциональное назначение. Программа выполняет три взаимоисключающие функции, которые выбираются набором аргументов командной строки.

Первая функция состоит в выводе справки о синтаксисе вызова, перечне опций и перечне поддерживаемых алгоритмов. Справка печатается, если аргументы отсутствуют или если где-либо в командной строке встретилась

опция `-h` либо `--help`. Справка имеет приоритет над любой другой опцией и над любой ошибкой в наборе опций, код возврата при этом нулевой.

Вторая функция состоит в создании ключа для выбранного алгоритма криптостойким генератором псевдослучайных чисел операционной системы и в записи этого ключа в файл или в стандартный поток вывода.

Третья функция состоит в шифровании или расшифровании потока данных. Источником входных данных служит файл или стандартный поток ввода, приемником результата служит файл или стандартный поток вывода. Ключ читается из файла либо из стандартного потока ввода или создается заново; созданный ключ сохраняется рядом с результатом.

Реализованы три алгоритма: простая замена, шифр Бофора и шифр Тритемия. Каждый размещен в собственной динамической библиотеке. Перечень имен алгоритмов и соответствующих им базовых имен библиотек задан в исходном коде исполняемого модуля; имя файла библиотеки формируется по базовому имени во время выполнения в зависимости от операционной системы. Дополнительные конфигурационные файлы для поиска библиотек не применяются.

Эксплуатационное назначение. Взаимодействие с пользователем происходит через интерфейс командной строки. Основной платформой является операционная система на базе ядра Linux версии 5.x и выше. Операционные системы Windows 10 и 11 заявлены как будущая платформа: участки, в которых обращения к операционной системе различаются, написаны для обеих систем, однако сборка и проверка под Windows не выполнялись.

Область применения. Программа предназначена для студентов и преподавателей и используется при изучении дисциплины «Программирование» и смежных дисциплин кафедры защиты информации. Программа обрабатывает файлы любых типов, включая исполняемые файлы и мультимедийные данные, без искажения содержимого, поэтому пригодна для демонстрации того, что криптографическое преобразование не зависит от формата данных.

Ограничение применения. Стойкость реализованных шифров недостаточна для защиты информации ограниченного доступа. Программа не является средством защиты информации и не проходила оценку соответствия требованиям к таким средствам.

3 ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1 Постановка задачи на разработку программы

Требовалось разработать консольное приложение, которое шифрует и расшифровывает данные произвольного типа алгоритмом, выбранным пользователем при запуске, и создает ключи для этих алгоритмов. Задача поставлена техническим заданием [1], ее условия перечислены ниже по группам.

Архитектура. Исполняемый модуль отделяется от реализаций алгоритмов. Каждый алгоритм размещается в отдельной динамически подключаемой библиотеке. Исполняемый модуль не вправе делать предположения о свойствах загруженного алгоритма и получает их через функцию метаданных. Взаимодействие строится по правилам двоичного интерфейса языка C; для передачи данных применяются структуры-обертки над указателем и размером буфера. Память для обмена выделяет исполняемый модуль, поэтому функции освобождения памяти в интерфейсе быть не должно.

Представление данных. Все обрабатываемые данные хранятся и передаются как массивы байт. Строковые и иные текстовые типы для хранения ключей и данных не применяются. Файлы и стандартные потоки открываются в двоичном режиме. Ключи создаются, читаются, сохраняются и передаются в двоичном виде.

Режимы работы. Программа работает в трех взаимоисключающих режимах: информационно-справочном, режиме генерации ключа и режиме криптографического преобразования данных. Набор аргументов проверяется до открытия файлов и до загрузки библиотеки; при недопустимом наборе печатается сообщение об ошибке, и работа прекращается.

Интерфейс. Синтаксис аргументов соответствует принятому в GNU. У каждой опции предусмотрены короткое имя из одного дефиса и одной латинской буквы и длинное имя из двух дефисов и слова.

Надежность. Исключения не должны покидать границу динамической библиотеки; об ошибке библиотека сообщает ненулевым кодом завершения. Исполняемый модуль перехватывает исключения, печатает сообщение об ошибке и завершается с ненулевым кодом. Отсутствие входного файла, недостаток прав, неверный ключ и отсутствие библиотеки обрабатываются как обычные ошибки, а не как аварийное завершение.

Защита данных. Ключи создаются только криптостойким генератором псевдослучайных чисел операционной системы. Буферы, в которых побывал ключ или открытый текст, принудительно перезаписываются перед освобождением, причем перезапись не должна удаляться оптимизатором.

Вычислительные ресурсы. При обработке файлов размером до 1 ГБ расход оперативной памяти не должен превышать 128 МБ.

Средства разработки. Язык C++ стандарта 2017 года, процедурный стиль, стандартные контейнеры и стандартные алгоритмы, сборка средствами CMake версии не ниже 3.10, максимальный уровень диагностики компилятора с трактовкой предупреждений как ошибок.

Тестирование. В составе проекта собирается отдельный исполняемый файл, предназначенный только для проверки библиотек алгоритмов. На каждый алгоритм приходится не менее трех проверок на разных входных данных.

Сопровождение. Подготавливаются сценарии установки и удаления для Linux и для Windows. Разработка ведется в системе контроля версий Git с отдельной веткой на каждую задачу и без принудительной перезаписи основной ветки.

Два условия технического задания выполнены в форме обоснованного отказа. Дополнение PKCS#7 предписано для случая реализации блочных шифров, а генерация вектора инициализации предписана для алгоритмов,

которые вектор используют. Все три реализованных шифра побайтовые: они не собирают данные в блоки фиксированного размера и не имеют состояния, которое требовалось бы задавать случайным значением в начале работы. Обоснование приведено в 3.3.

3.2 Описание алгоритма и функционирования программы

3.2.1 Состав программы и схема архитектуры

Программа состоит из исполняемого файла и трех динамических библиотек, по одной на алгоритм. Под Linux это `cryptum`, `libsubstitution.so`, `libbeaufort.so` и `libtrithemius.so`, под Windows это `cryptum.exe`, `substitution.dll`, `beaufort.dll` и `trithemius.dll`.

Исполняемый файл не содержит ни одного алгоритма шифрования. Ему известны только перечень имен алгоритмов, соответствующие им базовые имена библиотек и двоичный интерфейс, описанный в 3.2.2. Все, что относится к конкретному шифру, включая длину ключа, проверку ключа и его создание, находится в библиотеке.

Исходный код исполняемого файла разделен на семь модулей. Назначение модулей приведено в таблице 1.

Таблица 1 — Модули исполняемого файла

Модуль	Назначение
<code>main</code>	задает порядок работы, печатает справку, перехватывает исключения и формирует код возврата
<code>cli_options</code>	разбор набора аргументов командной строки и проверка его допустимости
<code>algorithm_registry</code>	перечень поддерживаемых алгоритмов, переход от имени алгоритма к базовому имени библиотеки
<code>crypto_module</code>	формирование имени файла библиотеки, загрузка библиотеки, получение пяти указателей на функции, владение дескриптором библиотеки
<code>byte_io</code>	двоичное чтение и запись, выбор между файлом и стандартным потоком, чтение и сохранение ключа
<code>pipeline</code>	цикл потоковой обработки данных и преобразование кодов завершения библиотеки в сообщения об ошибках

Модуль	Назначение
platform	настройка кодовой страницы консоли и двоичного режима стандартных потоков под Windows

Общий для исполняемого файла и всех библиотек код собран в статическую библиотеку `common`. В нее входят модуль `random_bytes`, обращающийся к криптостойкому генератору псевдослучайных чисел операционной системы, и модуль `secure_memory`, содержащий функцию затирания памяти и тип, владеющий затираемым буфером. Статическая библиотека собирается со скрытой видимостью символов, поэтому ее содержимое не попадает в таблицу динамических символов библиотек алгоритмов.

Схема архитектуры программы приведена на рисунке 1. На схеме показаны разделение на основной исполняемый модуль и динамические библиотеки, направления потоков данных и точки вызова функций двоичного интерфейса.

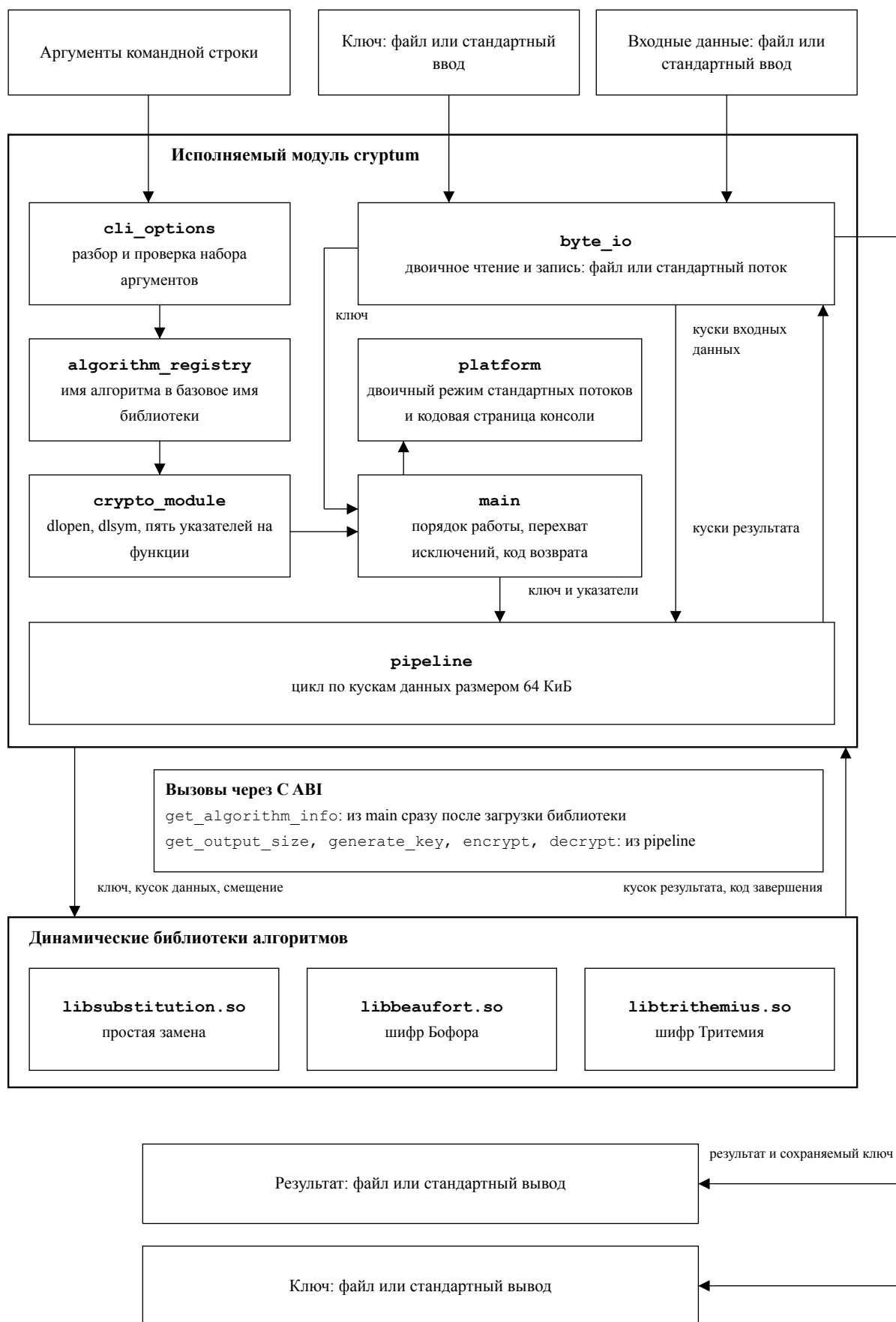


Рисунок 1 — Схема архитектуры программы

Порядок работы следующий. Функция `main` настраивает платформозависимые свойства консоли и стандартных потоков, после чего разбирает командную строку. Если аргументов нет или встречена опция вызова справки, справка печатается, и работа завершается с нулевым кодом. В противном случае имя алгоритма превращается в базовое имя библиотеки, библиотека загружается, и из нее извлекаются пять указателей на функции. Затем запрашиваются метаданные алгоритма, и под ключ объявленной длины отводится затираемый буфер. Ключ либо создается библиотекой, либо читается из файла или из стандартного потока ввода. Если задано сохранение ключа, ключ записывается. В режиме генерации ключа на этом работа заканчивается. Иначе открываются потоки входных и выходных данных, и выполняется цикл преобразования.

3.2.2 Двоичный интерфейс модулей шифрования

Заголовочный файл `include/crypto_abi.h` является общим для исполняемого файла и для всех библиотек алгоритмов. Все объявления в нем помещены в блок `extern "C"`, поэтому имена не подвергаются декорированию, а вызовы выполняются по соглашению языка C. Заголовок описывает структуры обмена, вид операции и коды завершения.

```
struct ConstBuffer { const uint8_t* data; size_t size; };
struct MutBuffer   { uint8_t* data;      size_t size; };
struct AlgorithmInfo { const char* algorithm_name; size_t
    key_size; };

enum OperationType { OPERATION_ENCRYPT = 0, OPERATION_DECRYPT =
    1 };

enum CryptoStatus {
    CRYPTO_OK                = 0,
    CRYPTO_ERROR_INVALID_KEY  = 1,
    CRYPTO_ERROR_INVALID_BUFFER = 2,
    CRYPTO_ERROR_RANDOM_FAILURE = 3,
    CRYPTO_ERROR_INTERNAL      = 4
};
```

Каждая библиотека алгоритма экспортирует ровно пять функций.

```
const AlgorithmInfo* get_algorithm_info(void);
size_t get_output_size(size_t input_size, int operation_type);
```

```
int generate_key(MutBuffer* key);
int encrypt(ConstBuffer key, ConstBuffer input, MutBuffer*
    output, uint64_t stream_offset);
int decrypt(ConstBuffer key, ConstBuffer input, MutBuffer*
    output, uint64_t stream_offset);
```

Ничего, кроме этих пяти имен, библиотека из собственного кода наружу не отдает. Вспомогательные функции каждой библиотеки помещены в безымянное пространство имен и обладают внутренним связыванием, а статическая библиотека `common` собрана со скрытой видимостью символов, поэтому ее содержимое в таблицу динамических символов библиотеки шифра не попадает. Помимо пяти перечисленных функций в этой таблице может оказаться слабый символ инстанцированного шаблона стандартной библиотеки: в `libsubstitution.so` это `std::iota`. К двоичному интерфейсу такой символ не относится и в глобальную область видимости процесса не попадает, поскольку библиотека открывается с флагом `RTLD_LOCAL`.

Функция `get_algorithm_info` возвращает указатель на статическую неизменяемую структуру с именем алгоритма и длиной ключа. Исполняемый модуль берет оттуда и имя, и длину и не хранит их у себя. Библиотека, вернувшая нулевой указатель, пустое имя или нулевую длину ключа, отвергается.

Функция `get_output_size` сообщает, сколько байт получается из `input_size` байт при указанной операции. Все три шифра сохраняют длину данных, поэтому все три возвращают `input_size`. Функция вызывается один раз перед началом цикла и задает размер выходного буфера.

Функция `generate_key` заполняет переданный буфер ключом. Создание ключа отдано библиотеке потому, что только алгоритму известно, какая последовательность байт является правильным ключом: для простой замены ключом служит перестановка всех 256 значений, а не произвольные 256 байт, и исполняемому модулю делать подобные предположения запрещено.

Функции `encrypt` и `decrypt` преобразуют один кусок данных. Параметр `stream_offset` содержит абсолютный номер первого байта куска в потоке, считая от нуля. Без этого параметра шифр, зависящий от положения

байта, невозможно применить к потоку по частям: продолжая обработку с середины файла, библиотека обязана знать, какой это по счету байт. Именно наличие такого параметра позволяет обрабатывать файл любого размера при постоянном расходе памяти.

Владение памятью. Всю память для обмена данными выделяет исполняемый модуль. Библиотеки буферов за вызывающего не выделяют, поэтому функции освобождения памяти в интерфейсе нет. При входе в функцию поле `size` структуры `MutBuffer` означает размер выделенного буфера, при выходе содержит число фактически записанных байт. Если места меньше требуемого, функция возвращает `CRYPTO_ERROR_INVALID_BUFFER` и буфер не изменяет.

Обработка ошибок. Тело каждой экспортируемой функции заключено в блок `try/catch`, поэтому за границу библиотеки не выходит ни одно исключение. Вызывающему передается код из перечисления `CryptoStatus`. Условия, при которых возвращается каждый код, приведены в таблице 2.

Таблица 2 — Коды завершения функций библиотеки алгоритма

Код	Условие
<code>CRYPTO_OK</code>	операция выполнена
<code>CRYPTO_ERROR_INVALID_KEY</code>	длина ключа не совпадает с объявленной либо ключ не удовлетворяет требованиям алгоритма
<code>CRYPTO_ERROR_INVALID_BUFFER</code>	нулевой указатель на структуру выходного буфера либо недостаточный размер выходного буфера
<code>CRYPTO_ERROR_RANDOM_FAILURE</code>	источник случайных чисел операционной системы недоступен
<code>CRYPTO_ERROR_INTERNAL</code>	прочие ошибки, в том числе нехватка памяти при построении обратной таблицы

3.2.3 Поточковая обработка данных

Преобразование выполняет модуль `pipeline`. Перед началом цикла создаются два буфера: входной размером 64 КиБ и выходной размером, который сообщила функция `get_output_size` для этого же числа байт и

выбранной операции. Оба буфера принадлежат типу, затирающему свое содержимое при разрушении.

Далее выполняется цикл. На каждом шаге из входного потока читается очередной кусок; если прочитано ноль байт, цикл завершается. Прочитанный кусок вместе с ключом, выходным буфером и текущим смещением передается функции библиотеки. Код завершения проверяется и при необходимости превращается в исключение с текстом ошибки. Число байт, которое библиотека записала в выходной буфер, выводится в выходной поток, после чего смещение увеличивается на число прочитанных байт. После выхода из цикла выходной поток сбрасывается, и его состояние проверяется еще раз.

Размер куска 64 КиБ выбран из двух соображений. С одной стороны, на гигабайт данных приходится 16384 обращения к вводу-выводу, что пренебрежимо мало по сравнению с самим обменом данными. С другой стороны, оба буфера вместе занимают 128 КиБ, помещаются в кеш процессора и не зависят от размера обрабатываемого файла.

Совпадение результата обработки одним вызовом и результата обработки того же потока по кускам разной длины проверяется отдельным тестом для каждого алгоритма.

Создание ключа выполняется похожим образом: под ключ объявленной длины отводится затираемый буфер, вызывается функция библиотеки, код завершения проверяется, и дополнительно проверяется, что библиотека сообщила ту же длину ключа, которая была объявлена в метаданных.

3.2.4 Реализованные алгоритмы шифрования

Сводка реализованных алгоритмов приведена в таблице 3.

Таблица 3 — Реализованные алгоритмы шифрования

Имя для опции -a	Имя в метаданных	Длина ключа, байт	Библиотека
substitution	simple substitution	256	libsubstitution.so
beaufort	beaufort	16	libbeaufort.so

Имя для опции -a	Имя в метаданных	Длина ключа, байт	Библиотека
trithemius	trithemius	16	libtrithemius.so

Все три реализованных шифра относятся к классическим: простая замена является одноалфавитным шифром замены, шифры Бофора и Тритемия многоалфавитными. Их построение и свойства описаны в [9].

Все три шифра работают побайтово и сохраняют длину данных. Далее P обозначает открытые данные, C обозначает результат шифрования, K обозначает ключ, i обозначает абсолютный номер байта в потоке, считая от нуля. Вся арифметика ведется по модулю 256 над беззнаковыми восьмибитными значениями, поэтому перенос через границу байта определен и совпадает со взятием остатка.

Простая замена. Ключом служит таблица из 256 байт, в которой для каждого возможного значения байта записано значение замены. Шифрование сводится к выборке из таблицы:

$$C[i] = K[P[i]]$$

Обратимость обеспечена тем, что таблица является перестановкой: каждое из 256 значений встречается в ней ровно один раз, поэтому у любого байта результата существует единственный прообраз. Расшифрование строит обратную таблицу K^{-1} , для которой $K^{-1}[K[b]] = b$, и применяет ее:

$$P[i] = K^{-1}[C[i]]$$

Ключ проверяется перед каждым преобразованием: подсчитывается число вхождений каждого из 256 значений, и ключ, в котором хотя бы одно значение встречается не ровно один раз, отвергается кодом `CRYPTO_ERROR_INVALID_KEY`. Преобразование не зависит от положения байта, поэтому смещение в потоке здесь не используется. Ключ создается заполнением таблицы значениями от 0 до 255 и последующим перемешиванием стандартным алгоритмом `std::shuffle`, источником случайных бит для которого служит генератор операционной системы.

Шифр Бофора. Ключом служит последовательность из 16 байт, которая мысленно повторяется вдоль всех данных. Каждый байт данных вычитается из соответствующего ему байта ключа:

$$C[i] = (K[i \bmod 16] - P[i]) \bmod 256$$

Преобразование является инволюцией: применение той же формулы к результату возвращает исходный байт.

$$P[i] = (K[i \bmod 16] - C[i]) \bmod 256$$

По этой причине функция `decrypt` просто вызывает `encrypt` с теми же аргументами.

Шифр Тритемия. Ключом служит последовательность из 16 байт. К каждому байту данных прибавляется байт того же повторяющегося ключа и, дополнительно, номер байта в потоке:

$$C[i] = (P[i] + K[i \bmod 16] + i) \bmod 256$$

$$P[i] = (C[i] - K[i \bmod 16] - i) \bmod 256$$

Слагаемое i приводит к тому, что одинаковые байты исходных данных переходят в разные байты результата, а сдвиг нарастает вдоль потока. Обратимость очевидна: при расшифровании вычитается ровно та же величина, а она однозначно определяется ключом и номером байта.

Для шифров Бофора и Тритемия правильным ключом является любая последовательность из 16 байт, поэтому создание ключа сводится к чтению 16 байт из генератора операционной системы.

3.2.5 Создание ключей и защита данных в оперативной памяти

Источник случайных чисел. Ключи создаются только криптостойким генератором операционной системы: под Linux это системный вызов `getrandom` [7], под Windows это функция `BCryptGenRandom`. Вызов, прерванный сигналом, повторяется; при любой другой ошибке выбрасывается исключение, которое библиотека превращает в код `CRYPTO_ERROR_RANDOM_FAILURE`. Генераторы псевдослучайных чисел стандартной библиотеки для создания ключей не применяются нигде.

Затирание памяти. Тип, владеющий буфером с ключом или открытым текстом, затирает буфер в деструкторе, поэтому затирание происходит на любом пути выхода, включая выход по исключению. Затирание выполняется функцией `explicit_bzero` [8] под Linux и функцией `SecureZeroMemory` под Windows; обе не могут быть удалены оптимизатором. Затираются буфер ключа в исполняемом модуле, входной и выходной буферы кусков данных и временная обратная таблица в библиотеке простой замены.

Права доступа к файлу ключа. Файл, создаваемый при сохранении ключа, получает права 0600. Права сужаются сразу после создания файла, пока он еще пуст, поэтому содержимое ключа не бывает доступно посторонним даже на время записи.

3.2.6 Обработка ошибок и коды возврата

Ошибки внутри исполняемого модуля передаются исключением. Перехват выполняется в единственном месте, в функции `main`. Текст ошибки печатается в стандартный поток ошибок в виде `cryptum: текст`, код возврата при этом равен 1. При успешном завершении код возврата равен 0. В стандартный поток вывода в режиме преобразования попадают только байты результата, диагностика туда не попадает никогда.

Коды завершения, полученные от библиотеки, превращаются в тексты ошибок в модуле `pipeline`. Текст включает название операции, поэтому по сообщению видно, на каком этапе произошел отказ. Полный перечень сообщений приведен в руководстве оператора.

3.3 Описание и обоснование выбора метода организации входных и выходных данных

Массивы байт вместо текста. Данные на всем пути от источника до приемника представлены как массивы беззнаковых восьмибитных значений. Внутри исполняемого модуля это стандартный вектор, на границе с библиотекой это структуры-обертки над указателем и размером. Текстовые типы не применяются ни для ключа, ни для открытых, ни для зашифрованных

данных. Причина состоит в том, что текстовые типы связаны с представлением текста: нулевой байт в середине данных обрывает строку с завершающим нулем, а любое преобразование кодировок изменяет байты. Для файла произвольного типа это означало бы искажение содержимого, тогда как техническое задание требует обработки файлов любых типов без искажений.

Двоичный режим потоков. Файлы открываются с признаком двоичного режима. Под Windows стандартные потоки ввода и вывода дополнительно переводятся в двоичный режим: иначе при выводе байт 0x0A превращался бы в пару 0x0D 0x0A, а при вводе байт 0x1A в середине данных прекращал бы чтение. Там же устанавливается кодовая страница консоли UTF-8, чтобы сообщения печатались правильно.

Единое обозначение стандартного потока. Если у опции входных данных, выходных данных или ключа указано значение -, используется соответствующий стандартный поток. Отсутствие опции входных или выходных данных равносильно указанию такого значения. Соглашение принято в утилитах Unix и позволяет включать программу в конвейер без дополнительных ключей.

Обработка кусками. Данные читаются, преобразуются и записываются кусками по 64 КиБ. Выбор потоковой схемы, а не полного чтения файла в память, продиктован требованием технического задания: при обработке файлов размером до 1 ГБ расход оперативной памяти не должен превышать 128 МБ. Полное чтение файла в память это требование нарушило бы уже на файле размером 200 МБ.

Потолок расхода памяти. Одновременно существуют входной буфер 64 КиБ, выходной буфер 64 КиБ и буфер ключа размером не более 256 байт. Ни одна из этих величин не зависит от объема обрабатываемых данных. Измеренный пиковый расход при обработке файла размером 1 ГиБ составил 3,2 МиБ и определяется в основном образом процесса и загруженными библиотеками, а не буферами. Подробности приведены в разделе 4.

Чтение ключа. Ключ читается ровно один раз, последовательно, без перемещения по потоку. Благодаря этому источником ключа может быть не только обычный файл, но и канал или подстановка процесса, например результат декодирования ключа из формата base64. Длина ключа проверяется приемом на один байт больше: после чтения ожидаемого числа байт делается попытка прочесть еще один байт, и если она удалась, ключ длиннее объявленной длины и отвергается. Так удастся отличить ключ нужной длины от более длинного, не выполняя позиционирования, которое для канала недоступно.

Запрет совмещения потоков. У стандартного потока ввода может быть только один потребитель, у стандартного потока вывода только один производитель. Поэтому одновременное чтение данных и ключа со стандартного ввода запрещено, как и одновременная запись данных и ключа в стандартный вывод. Обе проверки выполняются на этапе разбора аргументов, до открытия файлов и до загрузки библиотеки.

Отсутствие вектора инициализации. Вектор инициализации задает начальное состояние режима сцепления блоков. Ни один из трех реализованных шифров такого состояния не имеет: простая замена преобразует каждый байт независимо, а шифры Бофора и Тритемия используют вместо состояния номер байта в потоке, который вычисляется из смещения и известен обеим сторонам. Добавление вектора инициализации не изменило бы результат и лишь удлинило бы шифротекст. Требование технического задания сформулировано условно и относится к алгоритмам, которые вектор используют, поэтому оно выполнено.

Отсутствие дополнения PKCS#7. Дополнение выравнивает длину данных до кратной размеру блока. Все три реализованных шифра побайтовые и блока фиксированного размера не образуют, поэтому длина данных не обязана быть кратной чему-либо, а длина результата всегда равна длине входных данных. Требование технического задания относится к случаю реализации блочных шифров и потому неприменимо.

Решение является осознанным, а не пропуском. Интерфейс допускает добавление блочного шифра: функция вычисления размера выходного буфера сообщает длину результата отдельно для шифрования и для расшифрования, поэтому библиотека вправе удлинить данные при шифровании и укоротить при расшифровании, а вектор инициализации она обязана поместить в начало шифротекста сама, не привлекая исполняемый модуль.

Перезапись выходных данных. Выходной файл открывается с усечением, поэтому существующий файл перезаписывается без предупреждения. Состояние выходного потока проверяется после каждой записи и после сброса буферов: неудачная запись считается ошибкой, а не молчаливым обрывом данных.

3.4 Описание и обоснование выбора состава технических и программных средств

Язык программирования и стандарт. Язык C++ стандарта 2017 года [4] задан техническим заданием [1]. Из возможностей этого стандарта существенно используются работа с правами доступа к файлу средствами `std::filesystem` и функция `std::size` в тестовой утилите. Программа написана в процедурном стиле; полиморфизм в прикладной логике не применяется. Типы с деструкторами введены только там, где ресурс требуется освободить на любом пути выхода: это буфер, затирающий свое содержимое, и владелец дескриптора динамической библиотеки. Обработка данных в контейнерах выполняется стандартными алгоритмами `std::transform`, `std::for_each`, `std::all_of`, `std::iota` и `std::shuffle`; описание их поведения и требований к аргументам взято из справочника [5].

Система сборки. Выбрана CMake версии не ниже 3.10, как допускает техническое задание. Единственный файл `CMakeLists.txt` находится в корневом каталоге проекта и описывает шесть целей, перечисленных в таблице 4. Готовые файлы собираются в общий каталог, поэтому исполняемый файл и библиотеки оказываются рядом.

Таблица 4 — Цели сборки

Цель	Вид	Содержимое
common	статическая библиотека	обращение к генератору случайных чисел и затирание памяти
cryptum	исполняемый файл	основной модуль программы
substitution	динамическая библиотека	простая замена
beaufort	динамическая библиотека	шифр Бофора
trithemius	динамическая библиотека	шифр Тритемия
test_algorithms	исполняемый файл	проверка библиотек алгоритмов

Параметры компиляции. Для компиляторов семейства GCC и Clang заданы флаги `-Wall -Wextra -Wpedantic -Werror`, для MSVC заданы флаги `/W4 /WX /utf-8`. Тем самым включен максимальный уровень диагностики, и любое предупреждение прекращает сборку. Соответствие стандарту объявлено обязательным, расширения компилятора отключены. Проект собирается без единого предупреждения.

Механизм динамических библиотек. Под Linux применяются функции `dlopen`, `dlsym` и `dlclose` [6], под Windows функции `LoadLibraryA`, `GetProcAddress` и `FreeLibrary`. Имя файла библиотеки формируется во время выполнения из базового имени: под Linux получается `libX.so`, под Windows получается `x.dll`. Библиотека открывается с флагами `RTLD_NOW` и `RTLD_LOCAL`. Флаг `RTLD_NOW` выбран потому, что нужны все пять символов сразу и отказ должен обнаруживаться при загрузке, а не при первом вызове. Флаг `RTLD_LOCAL` не дает символам библиотеки попасть в глобальную область видимости процесса, что важно, поскольку у всех трех библиотек имена экспортируемых функций совпадают. Дескриптор библиотеки принадлежит типу, который закрывает его в деструкторе, поэтому библиотека выгружается на любом пути выхода.

Поиск библиотек во время выполнения. Исполняемый файл собирается со списком путей поиска, состоящим из одного элемента `$ORIGIN`, поэтому библиотеки ищутся в каталоге самого исполняемого файла. Настройка

переменных окружения не требуется, а установленная копия программы находит свои библиотеки независимо от текущего каталога.

Целевые платформы. Разработка и проверка велись на операционной системе на базе ядра Linux; поддерживаются архитектуры x86_64 и AArch64. Windows 10 и 11 заявлены как будущая платформа. Разделение на платформы выполнено условной компиляцией и затрагивает только четыре места: получение случайных байт, затирание памяти, загрузку динамической библиотеки и настройку консоли. Архитектура приложения при переносе не меняется.

Технические средства. Программе достаточно процессора архитектуры x86_64 или AArch64, 512 МБ свободной оперативной памяти и 64 МБ свободного места на диске. Фактический расход оперативной памяти много ниже, что показано в разделе 4.

Внешние зависимости. Кроме стандартной библиотеки C++ и интерфейсов операционной системы, внешних зависимостей нет. Тестовая утилита написана без применения сторонних библиотек тестирования: небольшой цикл проходит по списку именованных проверок и возвращает ненулевой код, если хотя бы одна из них не прошла. Такое решение принято потому, что подключение внешней библиотеки тестирования потребовало бы сетевого доступа при сборке и усложнило бы проверку работы на чистой системе.

Установка и удаление. Сценарий оболочки `scripts/install.sh` копирует исполняемый файл и три библиотеки в каталог `/usr/local/lib/cryptum` и создает символическую ссылку `/usr/local/bin/cryptum`, то есть помещает программу в каталог, который уже присутствует в переменной `PATH`. Сценарий `scripts/uninstall.sh` удаляет ровно то, что создала установка. Оба сценария требуют прав администратора, прерываются на первой ошибке и безопасны при повторном запуске. Для Windows подготовлены сценарии PowerShell

scripts/install.ps1 и scripts/uninstall.ps1, которые копируют файлы в каталог пользователя и правят пользовательскую переменную Path.

Система контроля версий. Разработка велась в Git. Основная ветка содержит только проверенное состояние, каждая задача выполнялась в отдельной ветке, созданной от основной, и вливалась обратно слиянием; принудительная перезапись основной ветки не применялась.

4 ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

Расчет экономической эффективности не производится ввиду учебного характера работы.

Ниже приведены измеренные показатели работы программы.

Условия измерения. Операционная система Ubuntu 24.04 LTS на базе ядра Linux, архитектура AArch64; компилятор GCC 13.3.0; сборка в конфигурации Release. Входными данными служил файл размером 1 ГиБ, то есть 1073741824 байта, заполненный случайными байтами. Данные читались из файла и записывались в файл на локальном накопителе. Время измерено как полное время выполнения процесса, расход памяти как максимальный размер резидентного набора. Результаты приведены в таблице 5.

Таблица 5 — Измеренные показатели на файле размером 1 ГиБ

Алгоритм	Время, с	Скорость, МиБ/с	Пиковый расход ОЗУ, МиБ
Простая замена	1,03	994	3,2
Шифр Бофора	0,77	1330	3,2
Шифр Тритемия	0,84	1219	3,2

Тот же файл, пропущенный через программу по конвейеру, то есть с чтением из стандартного ввода и записью в стандартный вывод, обработан шифром Бофора за 0,80 с, что соответствует скорости 1280 МиБ/с при том же расходе памяти. Правильность проверялась сравнением: расшифрованный файл побайтово совпал с исходным во всех трех случаях.

Техническое задание ограничивает расход оперативной памяти величиной 128 МБ при обработке файлов размером до 1 ГБ. Измеренный пик меньше этого предела примерно в сорок раз и не зависит от размера файла, поскольку одновременно в памяти находятся только два буфера по 64 КиБ и буфер ключа.

Сборка без явного указания конфигурации, то есть без оптимизации, дает на том же файле 2,92 с, что соответствует скорости 351 МиБ/с при неизменном расходе памяти.

Результаты тестирования. Тестовая утилита содержит 19 проверок, и все 19 проходят. Для каждого из трех алгоритмов выполняются шесть проверок: сверка с заранее выписанным по формуле результатом на коротких фиксированных данных, пустые входные данные, данные длиной 37 байт, не кратной длине ключа, один мегабайт псевдослучайных данных с фиксированным зерном, совпадение результата обработки одним вызовом и обработки по кускам длиной 1, 7, 13, 64, 300 и 615 байт, а также проверка длины созданного ключа и отказа при недостаточном размере буфера. Девятнадцатая проверка убеждается, что библиотека простой замены отвергает ключ, не являющийся перестановкой, и ключ неверной длины. Проверка по заранее выписанному результату нужна потому, что круговой прогон завершается успешно и при неверной, но обратимой формуле.

Трудоемкость разработки составила около двух недель работы одного исполнителя. Объем исходного текста программы составляет 1541 строку в 27 файлах, включая сценарий сборки, тестовую утилиту и сценарии установки.

5 ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

Перечень составлен по ГОСТ Р 7.0.5-2008 [3].

1. ГОСТ 19.201-78. Единая система программной документации.
Техническое задание. Требования к содержанию и оформлению.
Введ. 1980-01-01. М.: Стандартиформ, 2010.
2. ГОСТ 19.404-79. Единая система программной документации.
Пояснительная записка. Требования к содержанию и оформлению.
Введ. 1981-01-01. М.: Стандартиформ, 2010.
3. ГОСТ Р 7.0.5-2008. Система стандартов по информации,
библиотечному и издательскому делу. Библиографическая ссылка.
Общие требования и правила составления. Введ. 2009-01-01. М.:
Стандартиформ, 2008.
4. ISO/IEC 14882:2017. Programming languages — C++. Geneva: ISO,
2017.
5. C++ reference [Электронный ресурс]. URL:
<https://en.cppreference.com/w/cpp> (дата обращения: 19.09.2026).
6. dlopen(3) [Электронный ресурс] // Linux manual pages. URL:
<https://man7.org/linux/man-pages/man3/dlopen.3.html> (дата
обращения: 19.09.2026).
7. getrandom(2) [Электронный ресурс] // Linux manual pages. URL:
<https://man7.org/linux/man-pages/man2/getrandom.2.html> (дата
обращения: 19.09.2026).
8. explicit_bzero(3) [Электронный ресурс] // Linux manual pages. URL:
https://man7.org/linux/man-pages/man3/explicit_bzero.3.html (дата
обращения: 19.09.2026).
9. Menezes A. J., van Oorschot P. C., Vanstone S. A. Handbook of Applied
Cryptography. Boca Raton: CRC Press, 1996. 780 p.